Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Ingeniería en Sistemas de Información

Sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez

Entregable 7: Informe final académico

[Nombre del integrante 1] [Carné]
[Nombre del integrante 2] [Carné]
[Nombre del integrante 3] [Carné]

Arquitectura de Computadores II, sección 040
[Nombre del catedrático]
31 de octubre de 2026

Índice

Resumen 1

Sistema de monitoreo perimetral urbano para San Juan Sacatepéquez 2

- Objetivo general 2
- Objetivos específicos 2
- Cliente hipotético y alcance 2

Marco teórico 3

- Arquitectura orientada a eventos 3
- Correlación y procesamiento de eventos complejos 3
- Objetivos de nivel de servicio 3
- Privacidad de datos en ciudades inteligentes 4
- Registros de decisiones de arquitectura 4

Arquitectura propuesta 4

- Vista general 4
- Componentes 6
- Distribución por nodos y red 6
- Contrato de eventos y topics 7

Justificación técnica 7

Simulación de sensores urbanos 9

Modelo de correlación e inferencia 9

Visualización en tiempo real 10

Evaluación de resultados 14

- Correlación 14
- Latencia y capacidad 15
- Resiliencia 16

Calidad de datos y SLOs 16
Comparación con casos reales de ciudades inteligentes 16
Centro de Operaciones de Río de Janeiro 16
Barcelona 17
Detección acústica de disparos en St. Louis 17
Limitaciones y trabajo futuro 18
Conclusiones 18
Anexos 19
Referencias 20

Resumen

Este informe presenta el diseño, la justificación y la evaluación de una simulación funcional de un sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez, Guatemala. El sistema recibe eventos de cinco tipos de sensores simulados (botones de pánico, cámaras lectoras de placas, radares de velocidad, sensores acústicos y reportes ciudadanos), los valida, los transporta mediante Apache Kafka, los correlaciona con siete reglas de negocio que mantienen su estado en Redis, persiste eventos y alertas en PostgreSQL, avisa a la PNC, Bomberos, CONRED y una aseguradora, y los visualiza en Grafana. La arquitectura es orientada a eventos y distribuida a nivel lógico, desplegada on-premise en un solo servidor con Docker Compose, decisión justificada por el presupuesto limitado del cliente y la sensibilidad de los datos. Las decisiones se documentaron en doce Architecture Decision Records. La evaluación mostró que los nueve escenarios guionados generaron su alerta esperada, con latencias de 30 a 200 ms en tráfico normal; que el sistema sostiene unos 510 eventos por segundo cumpliendo el objetivo de latencia de 2 s, alrededor de 100 veces el tráfico normal estimado; y que la caída de un ingestor, del correlador o de Redis no provocó pérdida de eventos ni alertas duplicadas.

Palabras clave: arquitectura orientada a eventos, correlación de eventos, ciudades inteligentes, Apache Kafka, procesamiento en tiempo real

Sistema de monitoreo perimetral urbano para San Juan Sacatepéquez

Las ciudades generan cada día una gran cantidad de señales sobre lo que ocurre en sus calles: llamadas de emergencia, cámaras de tránsito, reportes de vecinos y alarmas. El valor de un centro de monitoreo no está en recibir esas señales por separado, sino en relacionarlas a tiempo para reconocer un incidente real y avisar a quien puede atenderlo. Este proyecto diseña y construye una simulación funcional de ese tipo de sistema para un cliente concreto, la Municipalidad de San Juan Sacatepéquez, y actúa como un equipo de arquitectura de software: cada decisión técnica relevante se respalda con un análisis de alternativas y una justificación comprensible para actores no técnicos.

Objetivo general

Diseñar, justificar y desarrollar una simulación funcional de un sistema de monitoreo perimetral urbano con capacidades de detección, procesamiento, correlación y visualización de eventos críticos, basado en una arquitectura distribuida.

Objetivos específicos

1. Simular el comportamiento, la frecuencia, la ubicación y el tipo de eventos de cinco clases de sensores urbanos, incluidos picos de carga y eventos inválidos.
2. Implementar una arquitectura orientada a eventos con ingesta balanceada, transporte en Kafka, correlación con estado en memoria, persistencia y avisos a instituciones externas.
3. Correlacionar múltiples fuentes para inferir incidentes como un posible robo en curso o un accidente vial, y distinguir los falsos positivos.
4. Visualizar en tiempo real mapas de calor, listas de incidentes, alertas críticas e indicadores de servicio.
5. Medir la latencia, la capacidad y el comportamiento ante fallas, y documentar las decisiones de arquitectura en ADRs.

Cliente hipotético y alcance

El cliente es la Municipalidad de San Juan Sacatepéquez, del departamento de Guatemala, con 218 156 habitantes (Instituto Nacional de Estadística [INE], 2019). Es

un municipio mediano, con presupuesto limitado y sin un equipo grande de tecnología, que necesita un centro de monitoreo propio. El sistema cubre siete sectores con coordenadas reales: Casco urbano, Sajcavillá, Cruz Blanca, Lo de Mejía, Ciudad Quetzal, Villas del Quetzal y Montúfar. Tres de ellos tienen zona escolar y cada uno tiene su propio límite de velocidad. No se usan sensores físicos: los sensores se emulan con programas en Python y con JMeter.

Marco teórico

Arquitectura orientada a eventos

En una arquitectura orientada a eventos, los componentes no se llaman directamente entre sí: publican hechos (eventos) en un intermediario y otros componentes los consumen cuando les corresponde. Esto desacopla a quien produce de quien consume, permite que varios consumidores reaccionen al mismo hecho y absorbe los picos de carga, porque los eventos esperan en el intermediario si un consumidor va más lento (Kleppmann, 2017). Apache Kafka implementa esta idea como un registro de mensajes distribuido, persistente y ordenado por partición, diseñado originalmente para procesar grandes volúmenes de registros de actividad (Kreps et al., 2011).

Correlación y procesamiento de eventos complejos

El procesamiento de eventos complejos busca patrones entre eventos simples para inferir situaciones de mayor nivel, normalmente dentro de ventanas de tiempo (Luckham, 2002; Etzion & Niblett, 2010). En este proyecto, un pánico, una cámara que ve un vehículo saliendo y un radar que mide exceso de velocidad en el mismo sector y dentro de una ventana de tiempo permiten inferir un posible robo en curso, aunque ninguno de esos eventos lo indique por sí solo.

Objetivos de nivel de servicio

Un objetivo de nivel de servicio (SLO) es un umbral medible de lo que el sistema promete, junto con lo que ocurre cuando no se cumple (Beyer et al., 2016). Para un sistema de alertas, la métrica más importante es la latencia entre el evento y la alerta: una alerta de robo pierde valor si llega minutos después.

Privacidad de datos en ciudades inteligentes

Las placas vehiculares y las ubicaciones son datos personales. La seudonimización reemplaza un dato identificable por un valor que no permite recuperarlo sin información adicional guardada aparte (Parlamento Europeo y Consejo de la Unión Europea, 2016). Un código de autenticación de mensajes con clave, como HMAC-SHA256 (Krawczyk et al., 1997), permite saber si dos lecturas corresponden a la misma placa sin conocer la placa. En Guatemala, la Ley de Acceso a la Información Pública reconoce la protección de los datos personales en poder de las instituciones públicas (Congreso de la República de Guatemala, 2008).

Registros de decisiones de arquitectura

Un Architecture Decision Record (ADR) es un documento breve que registra el contexto de una decisión, las alternativas, la elección y sus consecuencias (Nygard, 2011). El proyecto documenta doce ADRs en el Entregable 2 con el formato de la sección 4.1 del enunciado.

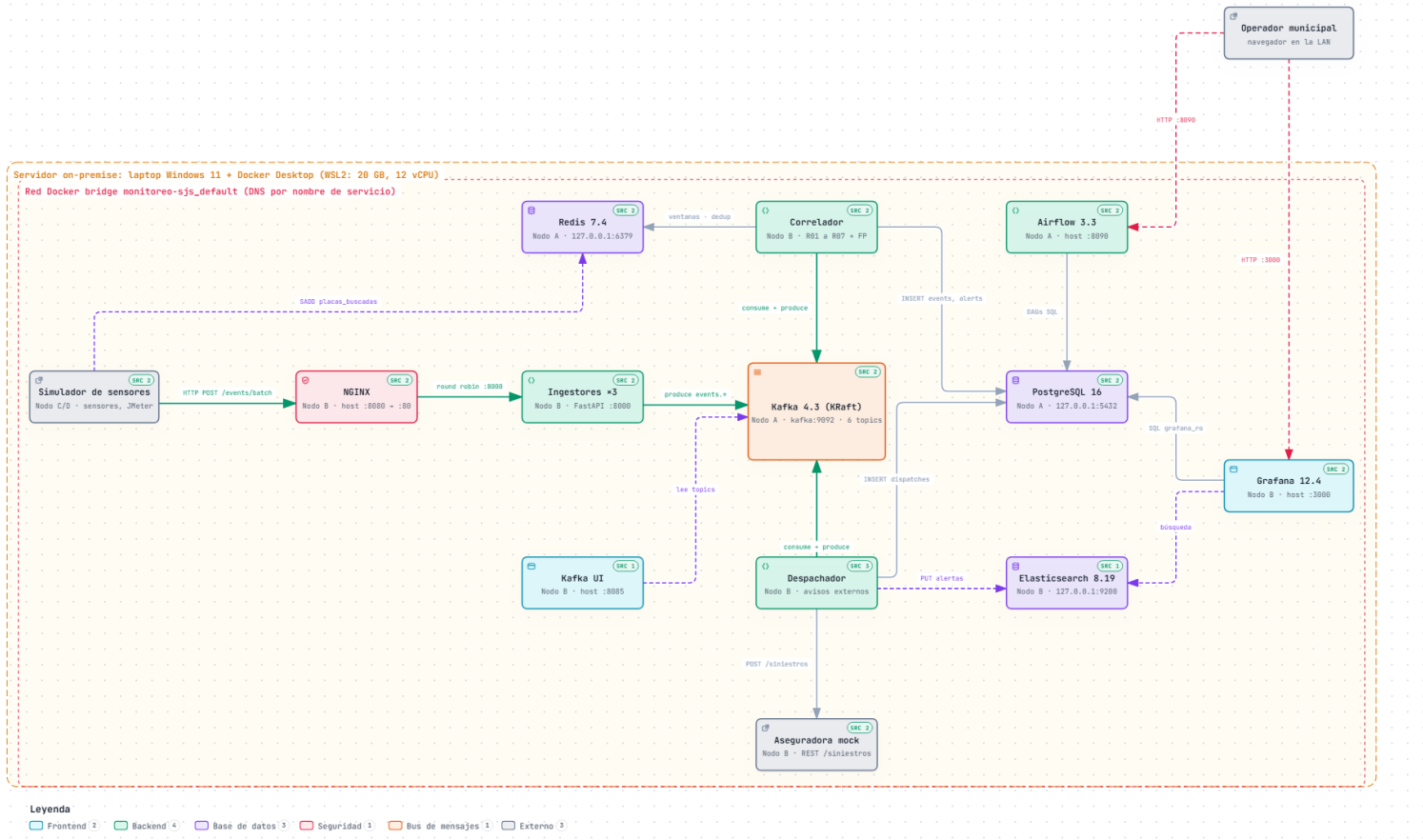
Arquitectura propuesta

Vista general

La Figura 1 muestra la infraestructura: nodos, servicios, red y puertos. El flujo principal es el siguiente. Los sensores simulados envían eventos en JSON por HTTP a NGINX, que los reparte en turnos entre tres ingestores. Cada ingestor valida el evento, rechaza los inválidos hacia una cola de mensajes muertos y publica los válidos en Kafka con el sector como clave. El correlador consume esos eventos, consulta y actualiza su estado en Redis, aplica las reglas, guarda eventos y alertas en PostgreSQL y publica las alertas en otro topic de Kafka. El despachador lee las alertas y las envía a la institución que corresponde, por Kafka o por REST, y las indexa en Elasticsearch. Grafana muestra todo en tiempo real y Airflow ejecuta las tareas periódicas.

Figura 1

Infraestructura del sistema: nodos, servicios y red



Nota. Elaboración propia a partir de la configuración del repositorio. Las flechas indican quién abre la conexión. Los nodos lógicos A, B y C/D de la sección 6 del enunciado se indican en cada componente; todos corren en un solo servidor físico (ADR-01). La versión interactiva está en diagrama-infraestructura.html.

Componentes

Tabla 1

Componentes del sistema

Componente	Tecnología	Función	Nodo
Simulador	Python, JMeter	Genera eventos de los cinco sensores, picos de carga, escenarios guionados y reportes manuales	C/D
Balanceador	NGINX 1.30	Única puerta de entrada (puerto 8080); reparte en turnos y tolera la caída de un ingestor	B
Ingestores (×3)	Python, FastAPI, Pydantic	Validan, estandarizan y publican en Kafka; responden 503 si Kafka no confirma	B
Broker	Apache Kafka 4.3 (KRaft)	Transporta y conserva los eventos 7 días en seis topics	A
Correlador	Python	Aplica las reglas R01 a R07 y detecta falsos positivos	B
Estado	Redis 7.4 (AOF)	Ventanas por sector, vehículos, duplicados y cooldowns	A
Base de datos	PostgreSQL 16	Eventos, alertas, envíos, inválidos, violaciones de SLO y estadísticas	A
Despachador	Python	Enruta alertas a PNC, Bomberos, CONRED y aseguradora; indexa en Elasticsearch	B
Aseguradora	Python, FastAPI	Entidad externa simulada que recibe accidentes por REST	B
Búsqueda	Elasticsearch 8.19	Búsqueda histórica de alertas	B
Visualización	Grafana 12.4	Cuatro dashboards: operador, SLOs, histórico e instituciones	B
Tareas periódicas	Apache Airflow 3.3	Retención de datos, reporte diario y análisis de falsos positivos	A

Distribución por nodos y red

El enunciado propone una distribución de referencia en nodos: el nodo A con Kafka, Airflow, PostgreSQL y Redis; el nodo B con los microservicios, Elasticsearch y Grafana; y los nodos C y D con los generadores de eventos. El proyecto respeta esa separación lógica, pero despliega los tres nodos en un solo servidor físico, porque es la opción adecuada para el presupuesto del cliente (ADR-01). Todos los contenedores comparten una red bridge de Docker y se encuentran por nombre de servicio. Hacia la red local solo se publican los puertos de uso humano: 8080 (entrada de eventos), 3000

(Grafana), 8085 (Kafka UI) y 8090 (Airflow). Las bases de datos y el puerto externo de Kafka solo aceptan conexiones de la misma máquina (127.0.0.1), y los servicios internos no publican ningún puerto.

Contrato de eventos y topics

Todos los eventos comparten un sobre basado en el ejemplo del catedrático, con los bloques device, location, data y metadata, un identificador único y dos marcas de tiempo: cuándo ocurrió el evento y cuándo se envió. Se usan seis topics organizados por etapa del flujo (Tabla 2). Las notas de diseño del grupo consideraron un topic por tipo de sensor, pero se descartó porque el correlador necesita recibir en orden, y por un solo canal, los eventos de distintos tipos de un mismo sector; la separación se hizo por etapa y por consumidor (ADR-09).

Tabla 2

Topics de Kafka

Topic	Particiones	Productor	Consumidor
events.standardized	3 (clave = sector)	Ingestores	Correlador
events.invalid	1	Ingestores	Despachador (tabla invalid_events)
correlated.alerts	3	Correlador	Despachador
external.pnc	1	Despachador	PNC
external.bomberos	1	Despachador	Bomberos
external.conred	1	Despachador	CONRED

Justificación técnica

Las decisiones de arquitectura se resumen en la Tabla 3 y se desarrollan en el documento de ADRs (Entregable 2, anexo a este informe), cada una con su contexto, alternativas, consecuencias, impacto en el negocio y evidencia medida.

Tabla 3

Resumen de las decisiones de arquitectura

ADR	Decisión	Razón principal
ADR-01	On-premise en un solo servidor	Costo bajo y fijo (inversión estimada de Q28 000 a Q45 000 frente a unos Q84 000 a Q120 000 al año en la nube), datos bajo control municipal y latencia sin depender del internet
ADR-02	Apache Kafka	Conserva y ordena los eventos; si un consumidor se cae, no se pierden
ADR-03	PostgreSQL, Redis y Elasticsearch	Relacional para auditoría; memoria rápida para correlacionar; búsqueda solo para el histórico
ADR-04	Airflow standalone	Historial y ejecución manual de las tareas, aunque a esta escala no se justifica por carga
ADR-05	Docker Compose sin Kubernetes	En un solo servidor Kubernetes no aporta alta disponibilidad y sí complejidad
ADR-06	Al menos una vez + deduplicación	Cero pérdidas ante caídas de componentes distintos de Kafka
ADR-07	HMAC de placas y retención	Una fuga no revela placas; datos borrados a los 30 días
ADR-08	SLOs: p95 < 2 s, < 2 % inválidos	Incumplimientos registrados y visibles, no solo dashboards
ADR-09	Topics por etapa, clave = sector	Orden por sector para la correlación
ADR-10	Reglas determinísticas	Explicables y auditables ante la dirección y el área legal
ADR-11	Solo Python	Todo el código es explicable por el grupo
ADR-12	JSON con schemaVersion	Legible y evolucionable sin servidores adicionales

Un punto que suele cuestionarse es por qué se considera distribuido un sistema que corre en una sola máquina. La respuesta es que cada componente es un proceso independiente, en su propio contenedor, que solo se comunica con los demás por la red (HTTP, Kafka, protocolos de PostgreSQL y Redis). Ninguno comparte memoria ni archivos con otro. Si la municipalidad compra un segundo servidor, basta mover contenedores y cambiar direcciones de red, sin modificar el código (Merkel, 2014). La decisión de no usar Kubernetes es consciente: en un solo nodo agrega un plano de control y muchos conceptos sin dar alta disponibilidad real (Burns et al., 2016).

Simulación de sensores urbanos

El simulador genera los cinco tipos de sensores con las tasas de la Tabla 4, con más tráfico vehicular en las horas pico (06:00 a 09:00 y 17:00 a 20:00) y más pánicos y disparos de noche (19:00 a 02:00). Cada sector recibe eventos según un peso proporcional a su población y los eventos se dispersan dentro de un radio alrededor del centro del sector, para que el mapa de calor sea realista. Entre 1 % y 2 % de los eventos se dañan a propósito (placa en claro, sector inexistente, hora futura y otros) para probar la validación. Las placas son sintéticas con formato guatemalteco y la cámara solo envía su HMAC y una versión parcial.

Tabla 4

Sensores simulados y tasas de generación

Sensor	Datos principales	Tasa normal	Tasa pico (×20)
Botón de pánico	Tipo (pánico, emergencia médica, incendio), origen (botón fijo o app)	1 por minuto	20 por minuto
Cámara LPR	Hash de la placa, placa parcial, velocidad estimada, entrada o salida, vehículo	3 por segundo	60 por segundo
Radar de velocidad	Velocidad, límite, dirección, tipo de anomalía	2 por segundo	40 por segundo
Sensor acústico	Disparo, explosión o vidrio roto; decibeles; probabilidad	2 por minuto	40 por minuto
Reporte ciudadano	Accidente, incendio o altercado; mensaje libre; origen	2 por minuto	40 por minuto

Además del tráfico normal y de pico, el simulador ejecuta nueve escenarios guionados que reproducen incidentes concretos, entre ellos los dos ejemplos de la sección 2.4 del enunciado: un pánico seguido de un vehículo que sale a 85 km/h (robo en curso) y un pánico sin eventos complementarios (falso positivo). Un script interactivo permite ingresar reportes ciudadanos a mano y JMeter genera carga HTTP configurable.

Modelo de correlación e inferencia

El correlador implementa los ocho pasos del diagrama de referencia: recibe el evento desde Kafka, busca el sector o el vehículo en Redis, obtiene el último estado,

correlaciona, calcula la velocidad cuando aplica, aplica las reglas, guarda el nuevo estado y, si corresponde, persiste y publica la alerta. Las siete reglas y la regla de falso positivo se resumen en la Tabla 5; su funcionamiento completo, el estado en Redis y las garantías ante fallas se documentan en el Entregable 5.

Tabla 5

Reglas de correlación

Regla	Alerta	Condición resumida	Severidad
R01	ROBO_EN_CURSO	Pánico + vehículo saliendo + exceso de velocidad	Crítica
R02	ATAQUE_ARMADO	Disparo con probabilidad ≥ 0.7 + pánico	Crítica
R03	ACCIDENTE_VIAL	Reporte de accidente + anomalía de velocidad, o dos reportes	Alta
R04	INCENDIO_CONFIRMADO	Pánico de incendio + reporte de incendio o explosión	Crítica
R05	VELOCIDAD_ENTRE_CAMARAS	Misma placa en dos cámaras a más de 80 km/h promedio	Media o alta
R06	ZONA_ESCOLAR	Más de 30 km/h en zona escolar en horario escolar	Media
R07	VEHICULO_REPORTADO	Placa en la lista de vehículos robados	Alta
FP	PANICO_NO_CONFIRMADO	Pánico sin confirmación al cerrar la ventana	Baja

Cada alerta incluye un score de confianza entre 0 y 1, calculado como la proporción de condiciones cumplidas multiplicada por el promedio de la confianza de los sensores que participaron, y la lista de eventos de origen, lo que permite rastrear cualquier alerta hasta el sensor. La velocidad entre cámaras se calcula con la fórmula de haversine (Sinnott, 1984). El correlador evita repetir alertas con cooldowns, no reutiliza eventos ya usados y descarta duplicados por identificador.

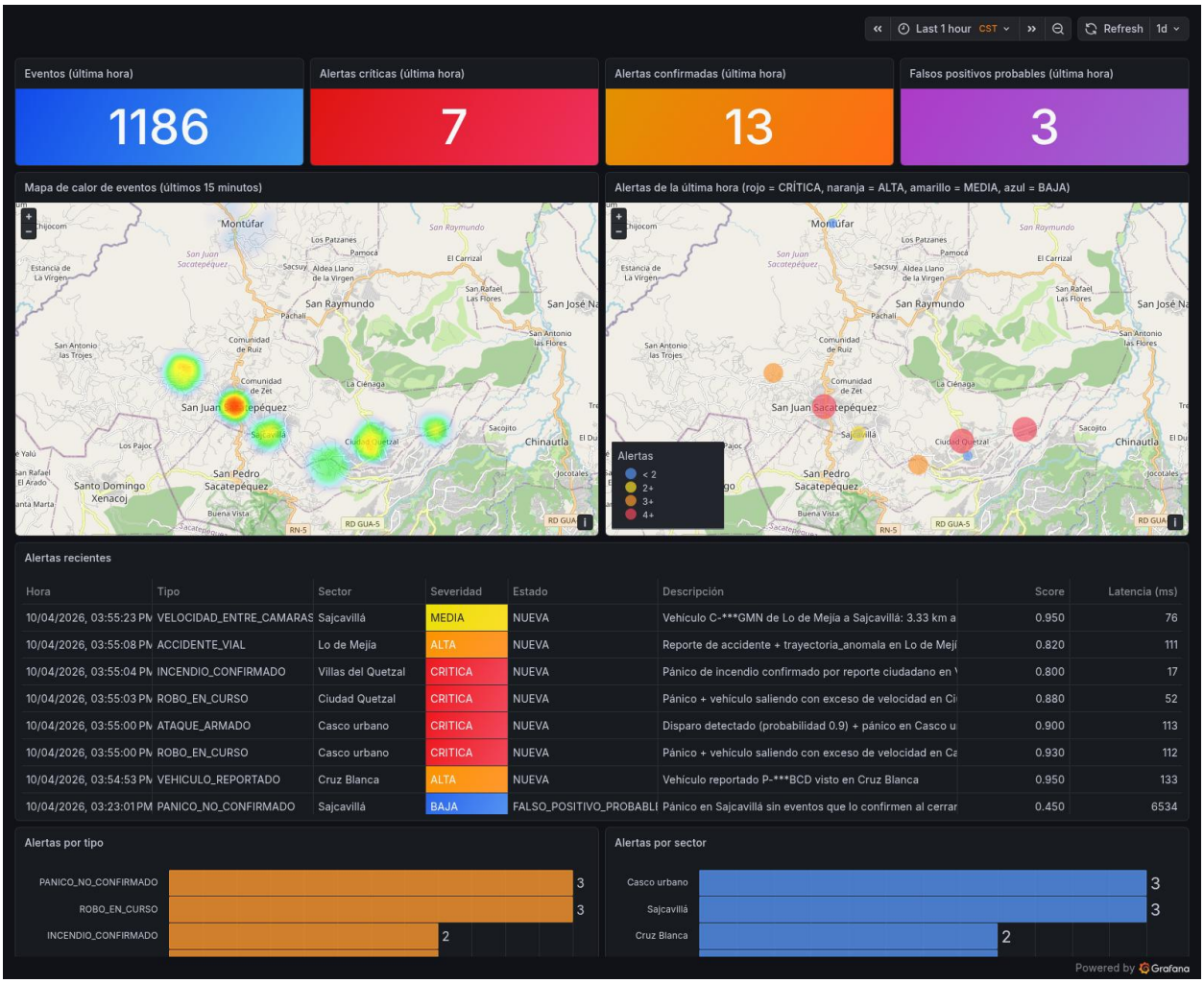
Visualización en tiempo real

Grafana se configura automáticamente desde archivos versionados, con un usuario de base de datos de solo lectura, refresco cada 5 segundos y hora de Guatemala. Se construyeron cuatro dashboards: el centro de monitoreo para el operador (Figura 2), operación y SLOs para el técnico (Figura 3), el histórico con

búsqueda en Elasticsearch y las estadísticas de Airflow (Figura 4), y la bandeja de instituciones, que muestra lo que recibió cada entidad externa (Figura 5).

Figura 2

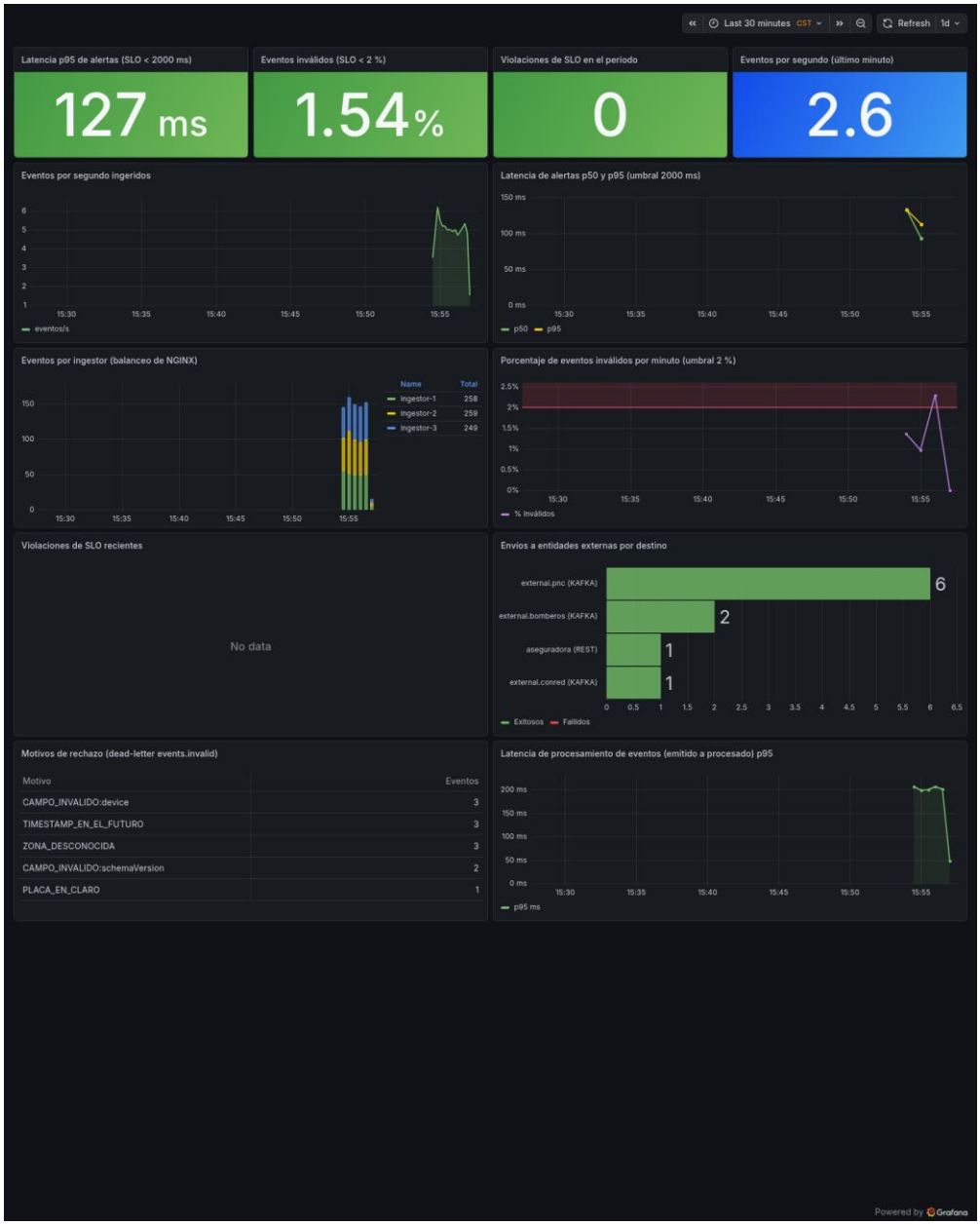
Dashboard 1: Centro de Monitoreo San Juan Sacatepéquez



Nota. Mapa de calor de eventos de los últimos 15 minutos, alertas coloreadas por severidad sobre el mapa del municipio, tabla de alertas recientes y contadores. Captura del sistema en funcionamiento.

Figura 3

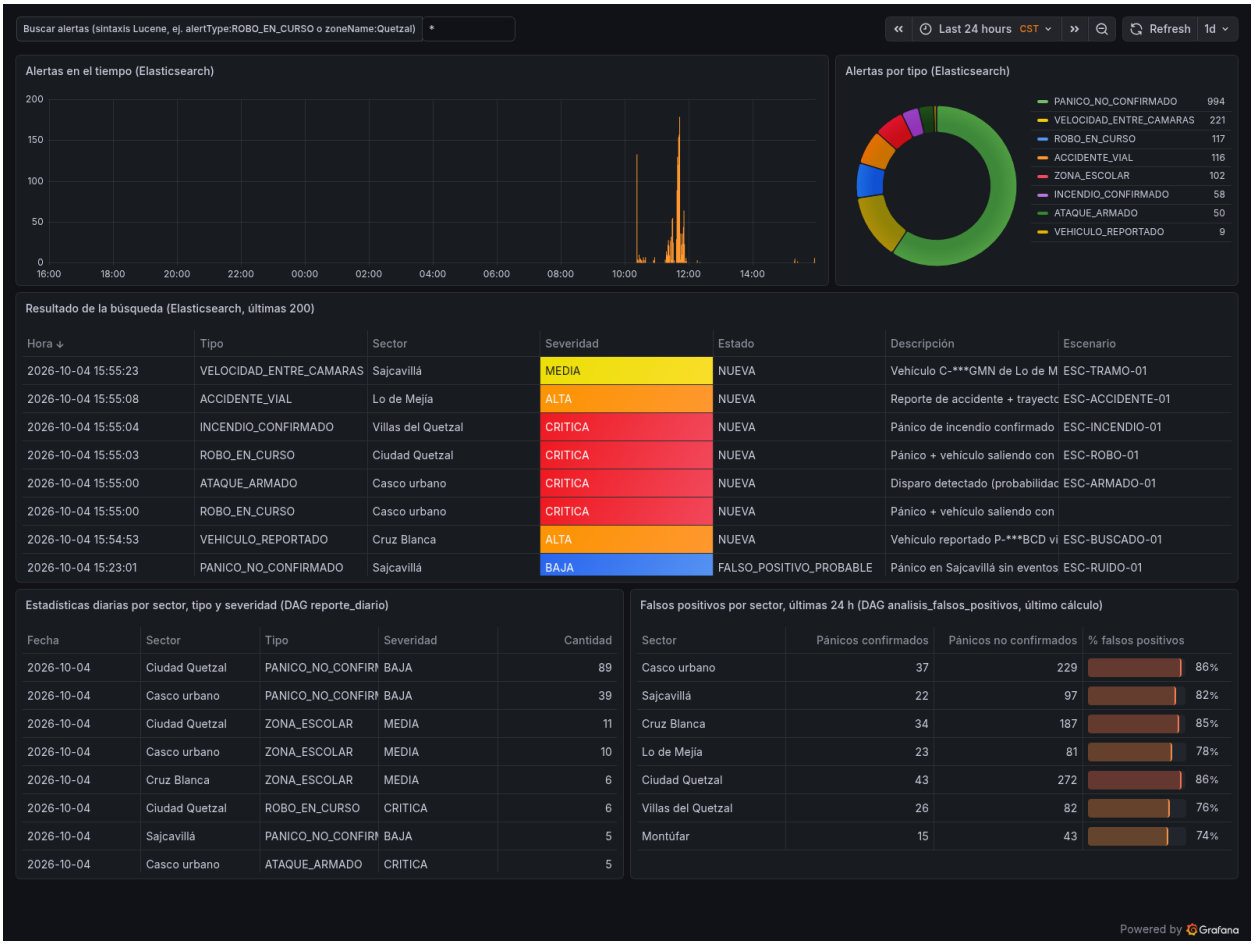
Dashboard 2: Operación y SLOs



Nota. Latencia p95 de alertas con el umbral de 2 000 ms, porcentaje de eventos inválidos, eventos por segundo, eventos por ingestor (balanceo de NGINX), violaciones de SLO y envíos por institución.

Figura 4

Dashboard 3: Histórico



Nota. Búsqueda de alertas en Elasticsearch, estadísticas diarias del DAG reporte_diario y proporción de falsos positivos por sector del DAG analisis_falsos_positivos.

Figura 5

Dashboard 4: Bandeja de instituciones



Nota. Alertas enviadas a cada institución, las más recientes primero, a partir de la tabla dispatches.

Evaluación de resultados

La evaluación se hizo en el servidor del proyecto (Intel i7-13650HX, 32 GB de RAM, 20 GB y 12 núcleos asignados a Docker) con el perfil completo del sistema. Se midieron la exactitud de la correlación, la latencia, la capacidad máxima y el comportamiento ante fallas.

Correlación

Los nueve escenarios guionados generaron su alerta esperada (9 de 9), con latencias de 30 a 185 ms en las reglas en tiempo real. Los dos falsos positivos se generaron al cerrarse su ventana. El escenario de robo produjo además una alerta de zona escolar, correcta porque su radar marcó 92 km/h en un sector escolar. También se verificó que el sistema es reproducible: al clonar el repositorio en una carpeta nueva

y seguir solo el manual, el sistema arrancó desde cero y los escenarios volvieron a generar sus alertas.

Latencia y capacidad

Tabla 6

Resultados de las pruebas de carga

Prueba	Eventos/s	Perdidos	Latencia de alertas p95	Cumple el SLO
Tráfico normal	5	0	menos de 200 ms	Sí
Pico ×100 durante 2 minutos	508	0	976 ms	Sí
Pico ×130 durante 90 s	661	0	13 668 ms	No
Pico ×160 durante 90 s	813	0	42 847 ms	No
Ingestión con JMeter (sin correlador)	26 500 o más	0	No aplica	No aplica

Nota. Cuando la carga supera la capacidad del correlador no se pierden eventos: esperan en Kafka y se procesan después, por lo que solo se incumple la latencia. El límite de la ingestión lo puso el generador de carga, no el sistema.

El máximo sostenido de extremo a extremo cumpliendo el objetivo de 2 segundos es de unos 510 eventos por segundo, alrededor de 100 veces el tráfico normal estimado para el municipio. El cuello de botella es el correlador, un proceso Python que consulta Redis varias veces por evento: con la carga máxima, el servidor completo usó menos de un núcleo de doce y unos 4.6 GB de 20 GB de memoria. Durante las pruebas se detectó que la primera versión del correlador procesaba solo 66 eventos por segundo, porque leía toda la ventana del sector con cada evento; la optimización descrita en el Entregable 5 la llevó a 660, diez veces más. El diagrama de referencia menciona más de 10 000 solicitudes por segundo: la capa de ingestión supera esa cifra, mientras que la correlación de extremo a extremo se queda en unos 510 eventos por segundo, suficiente para la escala del cliente.

Resiliencia

Tabla 7

Resultados de las pruebas de resiliencia

Falla provocada	Eventos perdidos	Alertas duplicadas	Observación
Caída de un ingestor (20 s)	0	0	NGINX reintentó en otro ingestor; el cliente vio un solo reintento
Reinicio del correlador	0	0	Apagado ordenado; de vuelta en menos de 1 s
Correlador detenido con 15 000 eventos en cola	0	0	Kafka los conservó y se procesaron al volver
Reinicio de Redis	0	0	Reconexión automática; latencia máxima de 3.2 s
Kafka caído 32 s	0	0	Los ingestores respondieron 503 y el cliente reintentó
Kafka caído 70 s	unos 300 de 13 215 (2.3 %)	0	Algunos lotes agotaron sus cinco reintentos

Kafka es el único componente cuya caída prolongada puede perder eventos. Para eliminar esa pérdida, los sensores reales deberían guardar los eventos y reenviarlos al recuperarse la conexión.

Calidad de datos y SLOs

El porcentaje de eventos inválidos se mantuvo entre 1.4 % y 1.5 %, por debajo del umbral de 2 %, y todos correspondieron a los daños introducidos a propósito por el simulador, lo que confirma que la validación no rechaza eventos correctos. Cada violación del SLO de latencia quedó registrada en la tabla slo_violations y visible en el dashboard 2.

Comparación con casos reales de ciudades inteligentes

Centro de Operaciones de Río de Janeiro

En 2010, Río de Janeiro inauguró un centro de operaciones desarrollado con IBM que integra en una sola sala la información de decenas de agencias municipales, entre ellas video de tránsito, lluvias, accidentes y emergencias, para coordinar la respuesta de la ciudad (Singer, 2012; C40 Cities, s.f.). Nuestro sistema sigue la misma

idea de integrar fuentes heterogéneas en un centro de monitoreo con visualización unificada y avisos a varias instituciones, pero a una escala y un costo adecuados para un municipio mediano: un solo servidor y software de código abierto en lugar de una instalación dedicada y contratos con un proveedor.

Barcelona

La experiencia de Barcelona muestra que una ciudad inteligente no depende solo de sensores y plataformas, sino de la gobernanza: de la coordinación entre áreas municipales, de una estrategia clara y de la capacidad de la institución para sostener la tecnología (Gascó-Hernández, 2018). Esta lección respalda dos decisiones del proyecto: priorizar componentes de código abierto que la municipalidad pueda operar sin depender de un proveedor (ADR-01 y ADR-05) y documentar cada decisión en lenguaje comprensible para la dirección (los ADRs).

Detección acústica de disparos en St. Louis

Mares y Blackburn (2012) evaluaron un sistema de detección acústica de disparos en St. Louis, Estados Unidos. Encontraron que el sistema no redujo de forma significativa la violencia y que, aunque bajaron las llamadas ciudadanas por disparos, el total de despliegues policiales por disparos aumentó de manera importante al sumarse las alertas automáticas. La lección es que un sensor aislado genera muchas alertas que consumen recursos de respuesta. Por eso, en nuestro sistema un disparo detectado no basta por sí solo para una alerta crítica: la regla R02 exige además una probabilidad de al menos 0.7 y un pánico en el mismo sector y en la misma ventana, y el escenario ESC-RUIDO-01 demuestra que un sonido de baja probabilidad termina como falso positivo y no moviliza a ninguna institución.

Tabla 8

Comparación con casos reales

Aspecto	Río de Janeiro	Barcelona	St. Louis	Este proyecto
Enfoque	Centro de operaciones integrado	Estrategia y gobernanza de ciudad inteligente	Sensor acústico aislado	Correlación de varias fuentes con visualización integrada
Escala y costo	Ciudad grande, instalación dedicada	Ciudad grande, programa municipal	Despliegue por zonas	Municipio mediano, un servidor y código abierto
Lección aplicada	Integrar fuentes y avisar a varias agencias	Gobernanza y capacidad propia	Evitar alertas por un solo sensor	Reglas multifuente, análisis de falsos positivos y ADRs

Limitaciones y trabajo futuro

- El servidor único es un punto de falla; se recomienda una UPS, respaldos diarios y, a futuro, un segundo servidor.
- Kafka con un solo broker no replica los datos; con dos servidores o más convendría un clúster de tres brokers.
- La regla R01 es sensible al ruido; conviene exigir que la anomalía de velocidad sea del mismo vehículo que salió.
- La clave de partición por sector desbalancea las particiones y limita la escalabilidad del correlador.
- La confianza de cada sensor debería calibrarse con datos reales de operación.
- Elasticsearch corre sin autenticación por ser laboratorio; en producción debe activarse su seguridad.

Conclusiones

Se diseñó y construyó una simulación funcional de un sistema de monitoreo perimetral urbano con detección, procesamiento, correlación y visualización de eventos críticos sobre una arquitectura orientada a eventos y distribuida a nivel lógico. La correlación de varias fuentes permitió inferir incidentes que ningún sensor indica por sí solo y distinguir los falsos positivos, y los nueve escenarios de prueba generaron su alerta esperada.

La decisión de desplegar on-premise en un solo servidor resultó adecuada para el cliente: con un costo bajo y los datos bajo control municipal, el sistema sostiene unas 100 veces el tráfico normal cumpliendo el objetivo de latencia, y no pierde datos ante la caída de sus componentes, salvo en una caída prolongada del broker. Las pruebas mostraron además el valor de medir: dos de los hallazgos más importantes, una espera innecesaria de un segundo en el correlador y una lectura de estado que lo hacía diez veces más lento bajo carga, solo aparecieron al medir la latencia y la capacidad.

Por último, los casos de Río de Janeiro, Barcelona y St. Louis confirman que el valor de una ciudad inteligente no está en acumular sensores, sino en integrarlos con criterio, gobernarlos con capacidad propia y evitar que generen más alertas de las que la ciudad puede atender, que fueron los principios que guiaron este proyecto.

Anexos

Este informe se acompaña de los demás entregables de la sección 8 del enunciado, que se entregan como documentos separados con el mismo formato:

- **Entregable 1.** Diseño de infraestructura: esquemático de nodos, servicios y segmentos de red (incluye el diagrama interactivo diagrama-infraestructura.html).
- **Entregable 2.** Documento de Architecture Decision Records: doce ADRs con el formato de la sección 4.1, anexo a este informe.
- **Entregable 3.** Configuración técnica de servicios y contenedores.
- **Entregable 4.** Scripts de simulación de eventos.
- **Entregable 5.** Documentación del sistema de correlación e inferencia.
- **Entregable 6.** Visualización en Grafana.
- **Repositorio.** Código fuente, configuración, scripts y manual de instalación (README): https://github.com/nglVlr/arqui2

Referencias

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM, 59*(5), 50–57. https://doi.org/10.1145/2890784

C40 Cities. (s.f.). *Rio Operations Center*. https://www.c40.org/case_studies/rio-operations-center

Congreso de la República de Guatemala. (2008). *Ley de Acceso a la Información Pública* (Decreto 57-2008).

Etzion, O., & Niblett, P. (2010). *Event processing in action*. Manning.

Gascó-Hernández, M. (2018). Building a smart city: Lessons from Barcelona. *Communications of the ACM, 61*(4), 50–57. https://cacm.acm.org/magazines/2018/4/226370-building-a-smart-city/fulltext

Instituto Nacional de Estadística. (2019). *XII Censo Nacional de Población y VII de Vivienda 2018*. https://www.censopoblacion.gt

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. En *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*.

Luckham, D. C. (2002). *The power of events: An introduction to complex event processing in distributed enterprise systems*. Addison-Wesley.

Mares, D., & Blackburn, E. (2012). Evaluating the effectiveness of an acoustic gunshot location system in St. Louis, MO. *Policing: A Journal of Policy and Practice, 6*(1), 26–42.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Parlamento Europeo y Consejo de la Unión Europea. (2016). *Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales*. Diario Oficial de la Unión Europea, L 119.

Singer, N. (2012, 3 de marzo). Mission control, built for cities. *The New York Times*.

Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope, 68*(2), 159.